



Operator Overloading :

As we know that function name can only include alphabet, number and underscore.

* But in C++ we can also use **operator** as a function name but we have to write **operator** before it.

Only valid C operators can be used as a name (function name).

Example 1:

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

Example 2:

```
int multiply(int a, int b) {
```

```
    return a * b;
```

```
}
```

Example 3:

```
int divide(int a, int b) {
```

```
    return a / b;
```

```
}
```

8 = d 8' = 0



Program: #include <iostream>

using namespace std;

class Complex

{

private:

int a, b;

public:

void setData(int x, int y)

{

a = x;

}

b = y;

void showData()

{

cout << "a = " << a << "b = " << b;

}

Complex add(Complex c)

{

Complex temp;

temp.a = a + c.a;

temp.b = b + c.b;

return(temp);

}

void main()

{

Complex c1, c2, c3;

c1.setData(3, 5);

c2.setData(5, 3);

c3 = c1.add(c2);

c3.showData();

}

Output :

a = 8 b = 8

Overloading of binary operator

Program: #include <iostream>
using namespace std;
class complex

{

private:

int a, b;

public:

void setData (int x, int y)

{
a = x;

b = y;

void showData ()

{
cout << "a = " << a << "b = " << b;

}

complex operator + (complex c)

→ using operator as function name

{

complex temp;

temp.a = a + c.a;

temp.b = b + c.b;

return (temp);

}

};

void main ()

{

Complex C1, C2, C3;

C1.setData (3, 4);

C2.setData (5, 6);

C3 = C1.operator + (C2);

C3 = C1 + C2;

→ We can also call it in this way also

* We did not need to use dot for calling operator name function

* And we did not need to use parenthesis for argument

* We can call simply like that C3 = C1 + C2; also

same thing

Binary operator mai jo left mai nota hai

vo hi collect object nota hai

And if are not using

• then we didn't need to

• write the operator

Overloading of unary operator:

↳ which take only **one operand**

Program: #include <iostream>

using namespace std;

class complex

{

private:

int a, b;

public:

void setData(int x, int y)

{

a = x;

b = y;

void showData()

{

cout << "a = " << a << " b = " << b;

Complex operator - ()

{

Complex temp;

temp.a = -a;

temp.b = -b;

return temp;

}

};

void main()

{

Complex c1, c2;

c1.setData(3, 4);

c2 = -c1; or c2 = c1.operator - ();

c2.showData();

}

C1 ne - operator ka call Kiya or iska return hmne C2

mai store Kiya.

alln : $(2) + 1 = 3$ tott xil pl/mis lloz no xil *



Overloading of unary operator ++ (pre and post)

```

program: #include <iostream>
using namespace std;
class member
{

```

```

private:

```

```

    int x;

```

```

public:

```

```

    void getData(int a)
    {

```

```

        x = a;
    }

```

```

    void showData()
    {

```

```

        cout << "x = " << x << endl;
    }

```

```

    member operator ++()
    {

```

// preincrement

* When you write pre-increment the compiler by-default call ++ function who doesn't take argument

```

        member i;

```

```

        i.x = ++x;

```

```

        return(i);
    }

```

```

    member operator ++(int)
    {

```

// postincrement

→ To help compiler to differentiate b/w pre increment and postincrement

```

        member i;

```

```

        i.x = x++;

```

```

        return(i);
    }

```

```

};

```

```

int main()
{

```

```

    member i1, i2;

```

```

    i1.getData(3);

```

```

    cout << "preincrement" << endl;

```

```

    cout << "before increment" << endl;

```

```

    i1.showData();

```

```

    i2 = ++i1; or i2 = i1.operator ++();

```

```

    cout << "After increment" << endl;

```

```

    i2.showData();

```

```

    i1.showData();
}

```

*

When you call/write post-increment the compiler by-default call ++ function who take one argument.



```
cout<<"postincrement"<<endl;
cout<<"beforeincrement"<<endl;
i1.showData();
i2 = i1++ ; or i2 = i1.operator++(0);
cout<<"After increment"<<endl;
i2.showData();
i1.showData();
}
```

Output :

preincrement

before increment

x = 3

After increment

x = 4

x = 4

postincrement

before increment

x = 4

After increment

x = 4

x = 5